Assignment 2

Deep Q-Networks

Advanced Algorithms

Johannes Graf
GitHub: Graf-J
Repository: MLDS-Q4-490-Advanced-Algorithms-HW2

November 2026

Contents

1 Repository and How to Run the Code **2**

2 Part (a): CartPole-v1 **3**

Method . . . 3

Hyperparameter Ladder . . . 3

Final Model (v5): Required Plots . . . 4

Robustness Across Seeds . . . 6

Non-Standard Plot: Phase Portraits . . . 6

Edge Cases . . . 7

Tests for Validating Correctness . . . 7

3 Part (b): MsPacman-v5 **11**

Method . . . 11

Training: Required Plots (i) and (ii) . . . 13

Model Selection and Required Plot (iii) . . . 13

Non-Standard Plot: Where Does Ms. Pac-Man Go? . . . 15

Edge Cases . . . 16

Tests for Validating Correctness . . . 17

4 Given an Additional 3 Months, What Other Validation Tests and Experiments Would I Perform? **19**

CartPole (a) . . . 19

Ms. Pac-Man (b) . . . 20

1 Repository and How to Run the Code

GitHub username Graf-J

Repository github.com/Graf-J/MLDS-Q4-490-Advanced-Algorithms-HW2 (public)

The project uses uv for Python and its dependencies (PyTorch with CUDA):

```
git clone https://github.com/Graf-J/MLDS-Q4-490-Advanced-Algorithms-HW2.git
cd MLDS-Q4-490-Advanced-Algorithms-HW2
uv sync             # creates .venv with the exact versions from uv.lock
uv run jupyter lab  # or open the notebooks in VS Code with the .venv kernel
```

Every part of the assignment is one Jupyter notebook that runs from top to bottom. All figures are written to `report/figures/`.

Part	Notebook	Content
a: CartPole	`cartpole-v1.ipynb`	Hyperparameter ladder, seed study, phase portraits, edge cases and validation tests (Section 2)
b: Ms. Pac-Man	`mspacman-v5.ipynb`	Training with crash recovery, model selection, evaluation, heatmaps, edge cases and validation tests (Section 3); the last cell opens a window in which the agent plays

Running without retraining. Every training run is saved to `checkpoints/a/<variant>/seed<k>/` (training log, network weights, config). With the default flag `RETRAIN = False`, the notebook loads a saved run instead of training it again, as long as its saved config equals the current one. A changed config is retrained automatically. All results in this report can therefore be reproduced from the repository without any training.

For part (b), the repository contains the training log, the network snapshots (every 500 episodes) and the cached evaluations in `checkpoints/b/dqn/`, but not the full checkpoint with the 7 GB replay buffer. The notebook recognises the finished run and loads it instead of training again. A new training (about 6 hours on a laptop GPU, 32 GB of RAM recommended) resumes automatically from the last checkpoint if it is interrupted.

How to run the validation tests. Every test described in this report is implemented in Section 7 *Validation Tests* of its notebook (part (a): Tests 1–9, part (b): Tests 1–10), each as a markdown cell followed by its code:

- The markdown cell explains **what** the test does and its **pass criterion**; why each test validates the algorithm is discussed in this report.
- The code cell runs the experiment and prints `[PASS]` or `[FAIL]` together with the measured numbers. A summary cell at the end of the notebook lists all checks.
- The tests run automatically with *Run All*. To rerun a single test, run all cells before Section 7 (they load the saved runs instead of training) and then the test's cells.
- Tests that need extra trainings: the control experiments and ablations of part (a) are saved in `checkpoints/a/` and loaded like all other runs. The $\gamma = 0$ run and the resume test of part (b) train small runs of a few minutes. Expensive evaluations of part (b) are cached in `checkpoints/b/dqn/cache/`; deleting this folder recomputes them.

In this report, a passed check is marked with ✓ and a failed one with **[FAIL]**.

Hardware. The networks were trained on a single GPU (NVIDIA GeForce GTX 1650 Max-Q, 4 GB) in a laptop with 32 GB of RAM.

2 Part (a): CartPole-v1

Method

A Double DQN learns $Q(s, a)$ for the two actions with the prescribed discount factor $\gamma = 0.95$.

- **Network:** MLP $4 \to$ hidden (ReLU) $\to 2$ Q-values, trained with Adam (learning rate $5 \cdot 10^{-4}$, batch size 32).
- **Replay buffer:** ring buffer of transitions $(s, a, r, s', \text{terminated})$, sampled uniformly. Learning starts once it holds 1,000 transitions; then one gradient step per environment step.
- **Target network:** Polyak-averaged copy of the online network, $\theta^- \leftarrow (1-\tau)\,\theta^- + \tau\,\theta$ after every step.
- **Double DQN target:** $y = r + \gamma\,(1 - \text{terminated})\,Q_{\theta^-}\big(s', \arg\max_{a'} Q_\theta(s', a')\big)$. The online network selects the next action and the target network evaluates it.
- **Exploration:** ϵ-greedy, decaying exponentially per episode from 1 to 0.01.
- **Training and evaluation:** 1,000 episodes, start states uniform in ± 0.15. The evaluation is greedy ($\epsilon = 0$) over 500 episodes, where episode i starts from seed i, so every network sees the same 500 start states.

Since Q is the discounted sum of rewards of $+1$, it is bounded by $1/(1-\gamma) = 20$, and after 100 steps it is already at 99.4 % of that bound. **The agent therefore only "sees" the next ∼60–100 steps.** This short horizon turns out to be the main difficulty of this part.

Hyperparameter Ladder

Starting from a deliberately simple baseline, I change **one hyperparameter at a time and keep all earlier changes**. Each variant is trained for 1,000 episodes with seed 0.

Variant	Change compared with the previous variant
v0: Baseline	32 hidden units, buffer 2,000, $\tau = 0.02$, MSE loss
v1: + buffer 50k	Buffer 2,000 $\to$ 50,000: the small buffer holds only the last few episodes and forgets failure states
v2: + $\tau = 0.005$	Slower target network
v3: + 128 hidden	Wider network
v4: + Huber & clip	Huber loss instead of MSE, gradient norm clipped at 10
v5: + reward shaping	Learning reward $r \cdot \max\big(0,\, 1 - (x'/2.4)^2\big)$, with x' the cart position after the step; best checkpoint on 20 validation start states (every 50 episodes)
Side: $\gamma = 0.99$	v4 with a longer horizon. Only an experiment, not a candidate for the final model

The shaped reward is 1 with the cart at the centre and 0 at the edge of the track. It is only used

for learning. **All reported rewards are the environment rewards.**

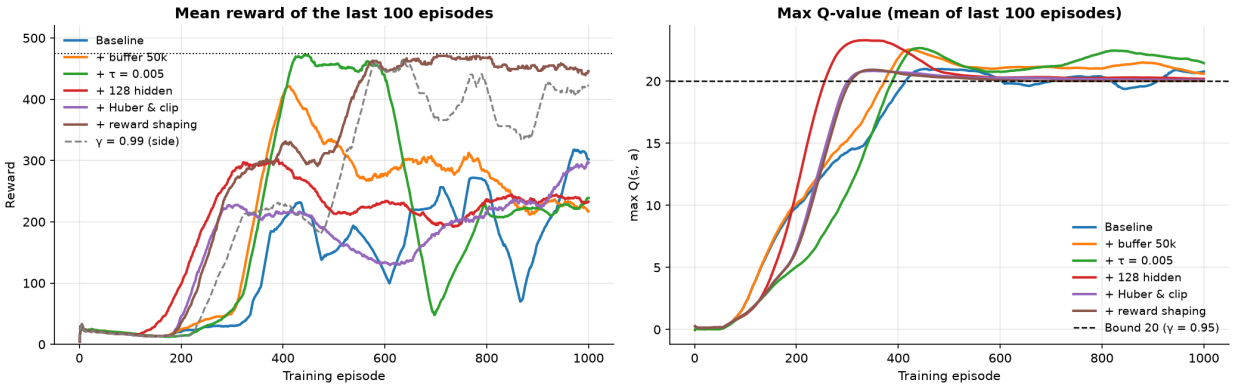


Figure 1: Moving average of the last 100 episode rewards (left) and of the maximum Q-value per episode (right) for all variants.

Variant	Mean reward over training	Mean of last 100	Evaluation mean ± std	Evaluation episodes = 500	Cart failures in training
v0: Baseline	149.9	302.0	215.1 ± 26.0	0 %	287
v1: + buffer 50k	212.3	217.3	253.5 ± 95.2	4 %	540
v2: + $\tau = 0.005$	211.5	238.9	450.2 ± 77.3	65 %	414
v3: + 128 hidden	202.8	232.8	213.4 ± 35.7	0 %	767
v4: + Huber & clip	172.6	296.4	500.0 ± 0.0	100 %	608
v5: + reward shaping	**323.4**	**445.9**	$\mathbf{500.0 \pm 0.0}$	**100 %**	**177**
Side: $\gamma = 0.99$	269.5	420.9	485.3 ± 78.8	97 %	317

- **v4 and v5 solve the task:** all 500 greedy evaluation episodes reach the time limit.
- **The ladder does not improve at every step.** v3 (the wider network alone) is worse than v2, and only pays off together with Huber loss and clipping in v4. With one seed, single steps are noisy (see *Robustness Across Seeds*).
- **Reward shaping has the largest effect on training:** compared with v4, the mean reward almost doubles (172.6 → 323.4) and cart failures drop from 608 to 177.
- **Max Q converges to the bound of 20** for every $\gamma = 0.95$ variant (and to $\approx 100 = 1/(1-0.99)$ for the side run).
- **No run reaches a moving average of 475 during training,** even though v5 is perfect in the evaluation. This is not caused by exploration: at the end, $\epsilon \approx 0.017$, and v5 still averages 494 with $\epsilon = 0.05$ (Test 7). Instead, the policy keeps changing with every gradient step: v5's validation drops to 151 at episode 400 and to 229 at episode 750 between perfect evaluations (Figure 2). **I therefore judge "solved" by the greedy evaluation of the final network.**

Final Model (v5): Required Plots

v5 has the highest evaluation mean, and the higher mean of the last 100 training episodes of the two perfect variants, so it is the reported model. Its best checkpoint is the final network of episode 1000 (the validation reached 20/20 several times, and ties prefer the later checkpoint).

(iii) Evaluation: mean = 500.0, std = 0.0 (Figure 3). Every episode is truncated at the time

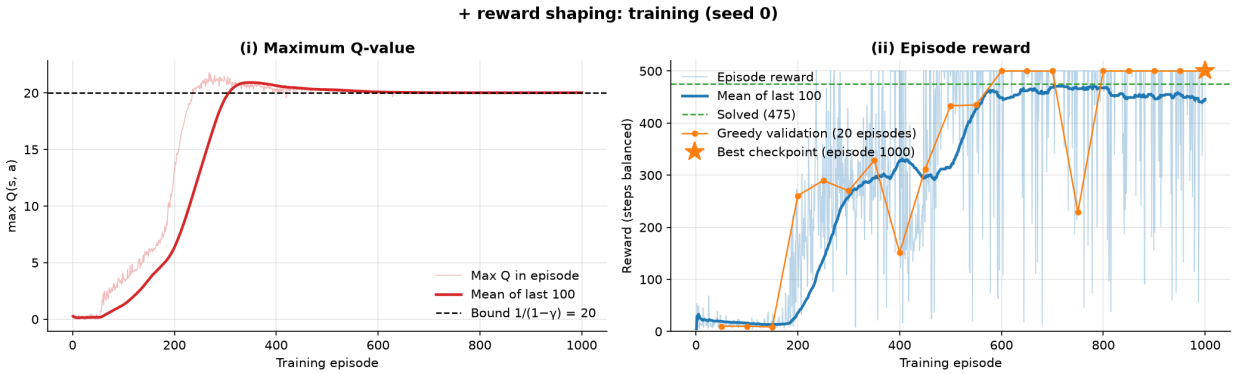


Figure 2: Training of v5: (i) maximum Q-value per episode and (ii) episode reward with the moving average of the last 100 episodes. Orange: greedy validation every 50 episodes; star: best checkpoint.

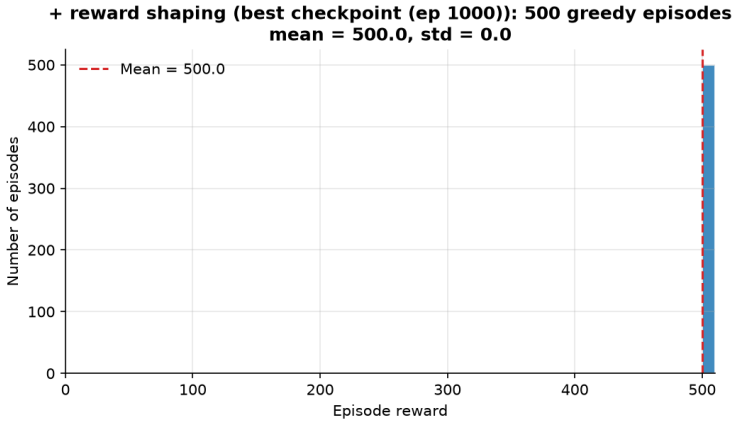


Figure 3: (iii) 500 greedy episodes of the final network of v5.

limit, so a std of 0 is the maximum possible result, not a bug.

Robustness Across Seeds

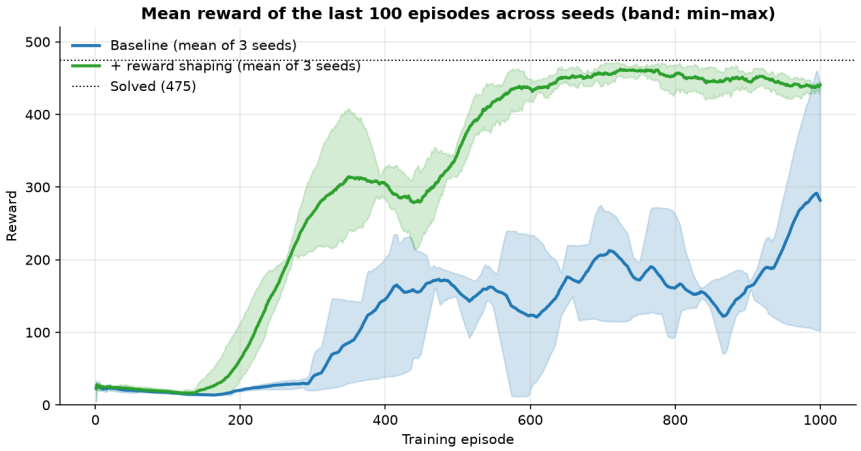


Figure 4: Moving average over 3 training seeds (band: minimum to maximum) for the baseline and v5.

Variant	Mean of last 100 (3 seeds)	Evaluation mean (seeds 0, 1, 2)	Seeds with evaluation = 500
v0: Baseline	281.9 ± 138.7	215.1, 13.8, 157.6	0/3
v5: + reward shaping	$\mathbf{440.8 \pm 4.7}$	**500.0, 500.0, 500.0**	**3/3**

v5 is robust: every seed reaches a perfect greedy policy, and the end of training is almost identical across seeds. The baseline is not. With seed 1 it collapses to 13.8, which is below the random policy (19.8).

Non-Standard Plot: Phase Portraits

Each panel shows one greedy episode of v5 from the **same start state** in the two phase planes of CartPole: pole angle vs. angular velocity (top) and cart position vs. cart velocity (bottom). The colour shows the time step, the red areas are the failure regions ($|\theta| > 0.21$, $|x| > 2.4$), and a red $\times$ marks a failure.

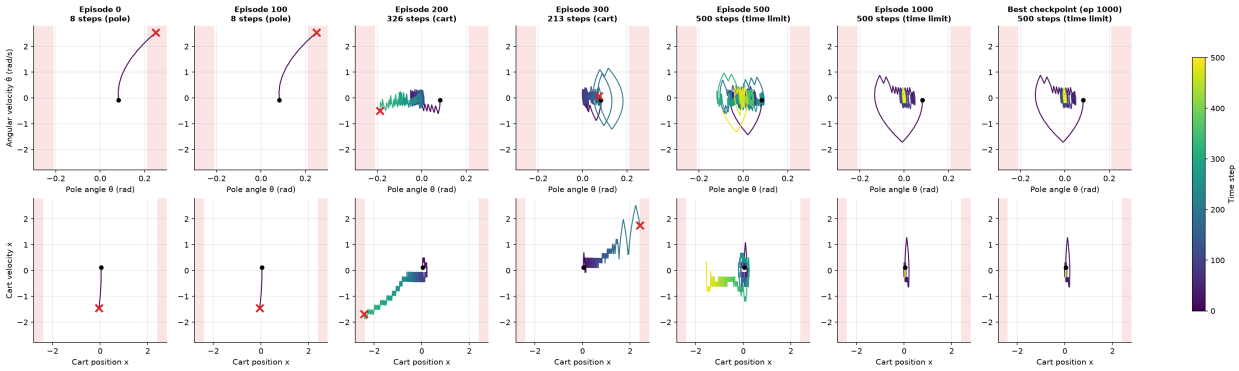


Figure 5: Phase portraits of greedy episodes of v5 at different points of training.

- **Episodes 0 and 100:** the trajectory runs straight into the failure region, and the pole falls after 8 steps.
- **Episodes 200 and 300:** the pole is already balanced (a tight zig-zag around $\theta = 0$), but the cart moves in a "staircase" at almost constant velocity towards the edge and leaves the track after 326 and 213 steps. **This is the short horizon of $\gamma = 0.95$ made visible:** the drift only causes a failure far beyond the ∼60 steps the agent sees.
- **Episode 1000:** after one large initial swing, the pole stays close to upright and the cart stays within $|x| < 0.3$ for all 500 steps.

Edge Cases

Edge case	Handling and check
Time limit (truncation at 500 steps)	Only *terminated* is stored as the done flag. A truncated state still has a future, so its last transition is bootstrapped. Checked: Q at the last step of truncated episodes is 19.99, not 1 (Test 1c).
Terminal mask (1 − terminated)	The target of a terminal transition is only r, since there is no next state. Checked with $\gamma = 0$ and the negated reward (Tests 2 and 3).
Overestimation of $\max_a Q$	Double DQN and a slow target network. Targets are computed without gradients, so only the online network is trained. Checked: Q stays at the bound of 20; without a target network it diverges (Test 4).
Replay warm-up	No update before the buffer holds 1,000 transitions. Waiting for a *full* 50k buffer would delay learning by more than 100 episodes.
Buffer too small	A 2k buffer only holds the last few episodes, so a good agent forgets the failure states (v0 → v1).
Short horizon ($\gamma = 0.95$)	A slow drift of the cart is barely punished. Addressed with reward shaping (v5), checked with the end reasons (Test 8) and the phase portraits.
Reward shaping must not make failing attractive	The shaped reward is clipped at 0, so it is ≥ 0 on the whole track: surviving longer never lowers the return, and Q stays ≤ 20.
Selecting the checkpoint on the test set	The best checkpoint is chosen on separate validation start states (seeds from 100,000 on). The evaluation uses seeds 0–499.
Saturated evaluation	500.0 ± 0.0 means "every episode reached the time limit". Harder evaluations still separate the variants (Test 7).
Moving average at the start	"Solved at" only counts full 100-episode windows.
Reproducibility	Python, NumPy and PyTorch are seeded, as is the first environment reset. Checked: training twice with the same seed gives identical rewards and weights; a different seed gives different rewards (Test 9 ✓).

Tests for Validating Correctness

All tests use the final networks from above unless stated otherwise. **17 of 21 checks pass.** The four failures share one cause, which I explain after the tests.

1. Q-values vs. actual discounted returns. For a state from which the episode terminates after k more steps, the true discounted return is known: $G = (1 - \gamma^k)/(1 - \gamma)$, i.e. exactly 1 one step before a failure and ≈ 20 for $k \geq 100$. To produce failures, I roll out the networks with random actions ($\epsilon \in \{0.1, 0.25, 0.5\}$) from wider start states (± 0.2) and compare the predicted $Q(s_t, a_t)$ with G (Figure 6).

Check	Baseline	v5
(a) Q one step before a failure < 5	14.10 [FAIL]	13.30 [FAIL]
(b) Correlation of Q and $G > 0.7$	0.78 ✓	0.49 [FAIL]
(c) Q at the last step of a truncated episode > 15	(no truncation)	19.99 ✓
(d) Long-lived states ($k \geq 100$): Q close to G	21.4 vs. 20.0 ✓	19.95 vs. 19.92 ✓

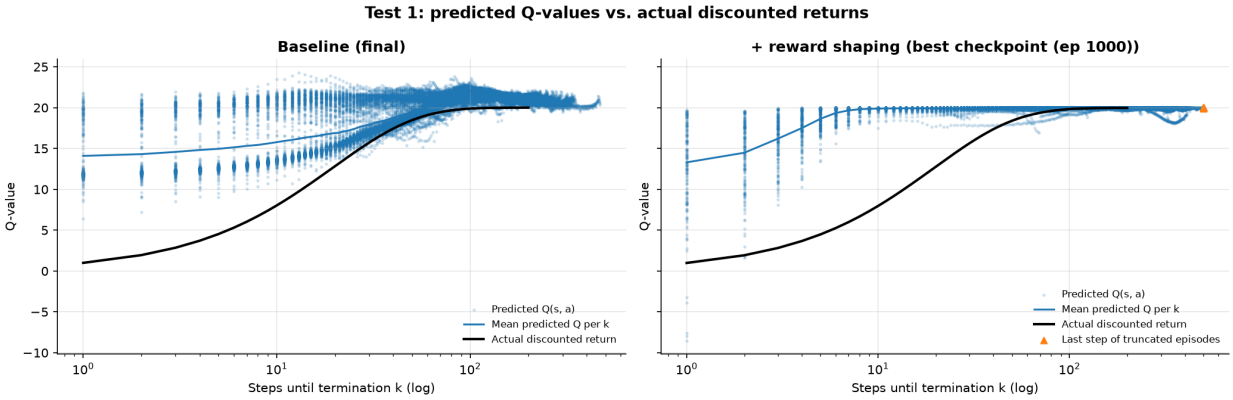


Figure 6: Predicted Q-values (blue) vs. the actual discounted return (black) as a function of the steps until termination.

Truncation is bootstrapped correctly and there is no runaway overestimation. However, **v5 only recognises an upcoming failure about 5 steps in advance**; before that, its Q stays at the plateau of 20. Check (b) is also partly limited by its design: Q estimates the return of acting *greedily* afterwards, whereas G is measured under the *noisy* policy. For $k > 1$, $Q > G$ is therefore expected. Only check (a) is independent of the policy.

2. $\gamma = 0$ sanity run. With $\gamma = 0$, the target is only the immediate reward, which is always 1, so a correct implementation must learn $Q = 1$ for every state and action. After 1,000 episodes, the mean $|Q - 1|$ over 1,112 states is **0.004**, and 95 % of the values are within ± 0.010 ✓. This confirms the target computation and the terminal mask in isolation from bootstrapping.

3. Control experiments. Trainings with the v5 configuration that only differ in the learning signal (1,000 episodes each):

Training	First 100 episodes	Last 100 episodes	Expected
Normal (v5)	16.1	445.9	Clearly increases ✓
Learning rate 0	16.9	9.7	Weights never change ✓
Negated reward (-1 per step)	19.4	7.6	Learns to fail fast, Q < 0 (mean -5.7) ✓
Reference: random policy	19.8		
Reference: always push left	9.2		

With the reward flipped, the same code learns the opposite behaviour and ends below the random policy. So the improvement comes from the learning signal, not from chance.

4. Ablation of the DQN components. v5 retrained as vanilla DQN (the target network selects and evaluates the next action) and without a target network ($\tau = 1$). I measure the overestimation at the start states: predicted $\max_a Q(s_0)$ minus the actual discounted return of the greedy episode.

Variant	Overestimation at s_0	Max Q (last 100)	Mean of last 100
Double DQN (v5)	+0.02	20.0	445.9
Vanilla DQN	+0.06	20.0	481.5
No target network	+3,272.5	15,810	9.7

- **The target network is essential** ✓: without it, Q diverges to about 790 times its bound and nothing is learned.
- **Double DQN overestimates less than vanilla DQN** ✓, but the difference is tiny and within the noise of one seed. With a slow target network, Q saturates at the bound, so there is little room for the maximisation bias.

5. The first 10 steps. One greedy episode of v5 from evaluation start state 0 (pole leaning left). The TD target $y = r + \gamma \max_a Q(s', a)$ is computed with the same network.

t	0	1	2	3	4	5	6	7	8	9
θ	−0.138	−0.141	−0.139	−0.132	−0.120	−0.103	−0.081	−0.054	−0.022	0.004
$\dot{\theta}$	−0.15	0.10	0.35	0.59	0.84	1.09	1.35	1.62	1.31	1.01
Q(L)	19.909	19.913	19.921	19.924	19.913	19.904	19.897	19.892	19.886	19.881
Q(R)	19.893	19.894	19.905	19.910	19.903	19.899	19.896	19.897	19.890	19.883
Action	L	L	L	L	L	L	L	R	R	R
Learning reward r	1.000	1.000	1.000	1.000	1.000	1.000	1.000	0.999	0.998	0.997

- **The agent pushes towards the side the pole leans to,** so the cart moves back under the pole. It switches to the right once the pole swings back quickly, and θ reaches 0 at $t = 9$.
- **The network is consistent with its own Bellman update:** the mean $|y - Q|$ over the 500-step episode is 0.001 ✓.
- **The action gap is tiny** (median $|Q(\text{R}) - Q(\text{L})| = 0.0016$). With $\gamma = 0.95$, one wrong action from a safe state costs almost nothing.
- The shaped reward falls below 1 as soon as the cart leaves the centre.

6. Does the network react correctly to the pole? I vary θ and $\dot{\theta}$ one at a time, with all other state variables at 0. Physically, the preference $Q(\text{R}) - Q(\text{L})$ must be negative at the negative end, positive at the positive end, and non-decreasing in between (Figure 7).

- **Angular velocity $\dot{\theta}$:** monotone and correctly signed, from −4.33 to +1.92 ✓.
- **Pole angle θ:** correctly signed (−0.34 / +0.10), but flat at ≈ 0 for $|\theta| < 0.18$, with a small decrease of 0.024 near $\theta = 0.2$ **[FAIL]**. With $\dot{\theta} = 0$, all these angles are recoverable with either action, so both Q-values are ≈ 20. The network decides mainly by $\dot{\theta}$.

7. Harder evaluation. The greedy evaluation is saturated at 500, so I also evaluate with 5 % random actions and from wider start states (±0.2):

Variant	Greedy, ±0.15	$\epsilon = 0.05$, ±0.15	Greedy, ±0.2
v4: + Huber & clip	500.0 ± 0.0	395.5 ± 151.6	500.0 ± 0.0
v5: + reward shaping	500.0 ± 0.0	$\mathbf{494.0 \pm 52.3}$	499.1 ± 21.0
Side: $\gamma = 0.99$	485.3 ± 78.8	397.4 ± 194.8	460.6 ± 123.3

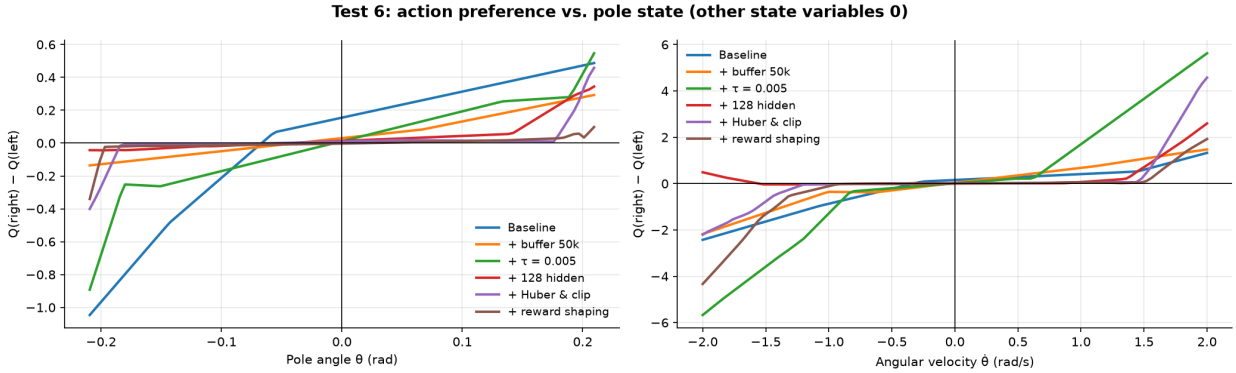


Figure 7: Action preference $Q(\mathrm{R}) - Q(\mathrm{L})$ vs. the pole angle (left) and the angular velocity (right), all other state variables 0.

v5 solves the standard evaluation ✓, and it is **by far the most robust to random actions**. v4 is perfect when greedy but brittle: random pushes move the cart, and without shaping it does not bring the cart back to the centre.

8. Why do episodes end? An episode ends when the pole falls, when the cart leaves the track, or at the time limit:

Variant	Training: pole / cart / time limit	Evaluation: pole / cart / time limit	Median max $\lvert x \rvert$ (evaluation)
v0: Baseline	601 / 287 / 112	0 / **500** / 0	2.41
v4: + Huber & clip	365 / **608** / 27	0 / 0 / 500	0.39
v5: + reward shaping	361 / 177 / 462	0 / 0 / 500	**0.15**
Side: $\gamma = 0.99$	348 / 317 / 335	17 / 0 / 483	1.74



Figure 8: How training episodes end, per block of 100 episodes.

- **With $\gamma = 0.95$, balancing the pole is easy; keeping the cart on the track is the actual difficulty.** The baseline loses *every* evaluation episode by the cart leaving the track while the pole stays up.
- **Both fixes of the short horizon reduce the cart failures** compared with v4: $\gamma = 0.99$ to 317 ✓ and reward shaping to 177 ✓. Only reward shaping also keeps the cart centred.

9. Reproducibility. See the edge cases above ✓.

Why four checks fail (Tests 1a, 1b and 6). The update rule itself is correct, as Tests 2, 3, 4a and 5 confirm. **The Q-function is only accurate where the replay buffer has data.** A good agent rarely visits states close to a failure, so there the network extrapolates its plateau value of 20. Together with the tiny action gaps of $\gamma = 0.95$, this produces the

high Q-values just before a failure (Test 1a) and the flat, slightly non-monotone response to θ (Test 6). Since the greedy policy never gets close to these states, this does not hurt performance, but it explains why v4 is brittle under random actions (Test 7).

Summary of Part (a)

- **Final model (v5):** Double DQN, 128 hidden units, buffer 50k, $\tau = 0.005$, Huber loss with clipping, and reward shaping towards the centre. It reaches **500.0 ± 0.0** in the greedy evaluation for all 3 seeds and is the most robust to random actions.
- **Main insight:** with $\gamma = 0.95$, the agent only sees ∼60 steps ahead, so the cart slowly drifting off the track is the dominant failure. Reward shaping fixes this best.
- **Validation:** 17 of 21 checks pass. The target network is essential, and the target computation is exact. The failed checks reveal that Q-values are inaccurate in rarely visited near-failure states.

3 Part (b): MsPacman-v5

Method

A Double DQN with a dueling head learns Ms. Pac-Man from the game images with the prescribed $\gamma = 0.99$ and all 9 actions. One training run takes about 6 hours, so **there is no hyperparameter ladder**. Instead, a single run (seed 0) combines the insights of part (a) with the standard Atari DQN settings.

Component	Choice and reason
Environment	`ALE/MsPacman-v5` with its defaults: frameskip 4 (one *agent step* = 4 frames), sticky actions with probability 0.25, time limit 108,000 frames.
State	The assignment's `preprocess_observation` (88×80 int8, divided by 128), **4 stacked frames**. A single frame shows neither the direction of movement nor the ghosts, which are not drawn in every frame (Figure 9).
Network	Nature CNN (32@8×8/4, 64@4×4/2, 64@3×3/1) with a **dueling head**: separate streams of 512 units for the state value $V(s)$ and the advantages $A(s,a)$, combined as $Q = V + A - \text{mean}_a A$ (2.8M parameters). In most states of Ms. Pac-Man, the actions are almost equivalent (part (a) already showed tiny action gaps), so learning V separately is cheap and helps.
Algorithm	Double DQN target, Huber loss, gradient norm clipped at 10, Adam with a constant learning rate of 10^{-4} ($\epsilon_{\text{Adam}} = 1.5 \cdot 10^{-4}$), batch 32. Of the advanced techniques the assignment suggests, Huber loss and an ϵ scheduler are used; a learning-rate scheduler is not, since the slow target network already keeps the training free of divergence.
Target network	Polyak update with $\tau = 0.001$ after every gradient step (time constant $\approx$ 4,000 agent steps). Part (a): the slow target network was essential.
Replay buffer	**1M transitions**, stored as single int8 frames (7 GB instead of 28 GB for stacked states). Part (a): a large buffer prevents forgetting. Uniform sampling.
Updates	50,000 random transitions before the first update, then one gradient step every 4 agent steps.
Rewards	Learning reward **clipped to sign**$(r) \in \{0, 1\}$: a dot gives 10 points, a ghost up to 1,600, so the raw scale differs by 160×. Q is therefore bounded by $1/(1-\gamma) = 100$. **All reported scores are raw game scores.**
Lives	A lost life is **terminal for the TD target** (no bootstrapping through a death), but the game continues. One episode is one full game with 3 lives.
Exploration	ϵ-greedy per agent step, linear from 1.0 to 0.1 over the first 1M steps and to 0.01 at 3M steps.
Length	6,000 episodes (the upper end of the assignment's range).

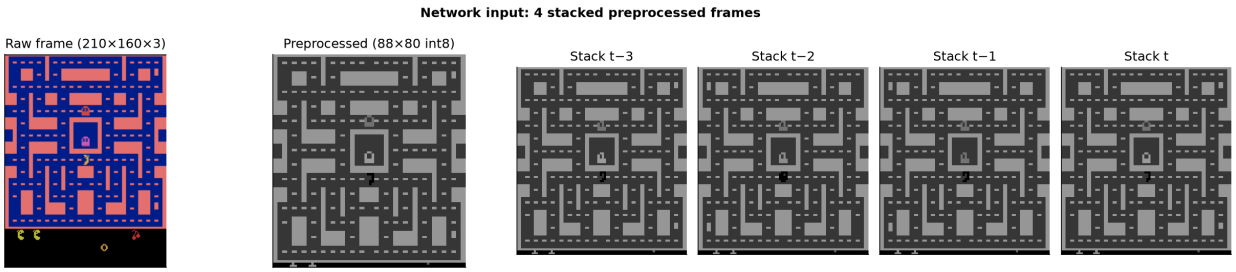


Figure 9: Raw frame, preprocessed frame and the stack of 4 frames the network sees. Ms. Pac-Man is set to the contrast value (black); the ghosts flicker between frames.

Crash recovery. Because the run lasted a night, every 250 episodes a full checkpoint (networks, optimiser, replay buffer, all random-generator states, log) is written to a new folder and then swapped in by renaming, so a crash while writing never destroys the last good checkpoint. Every episode starts from its own seed and the GPU runs in deterministic mode, so a resumed run is **bit-identical** to an uninterrupted one (Test 10).

Training: Required Plots (i) and (ii)

The training took **5.8 hours for 4.46M agent steps** (17.8M frames, ≈ 213 agent steps per second) on a laptop GPU (GTX 1650 Max-Q).

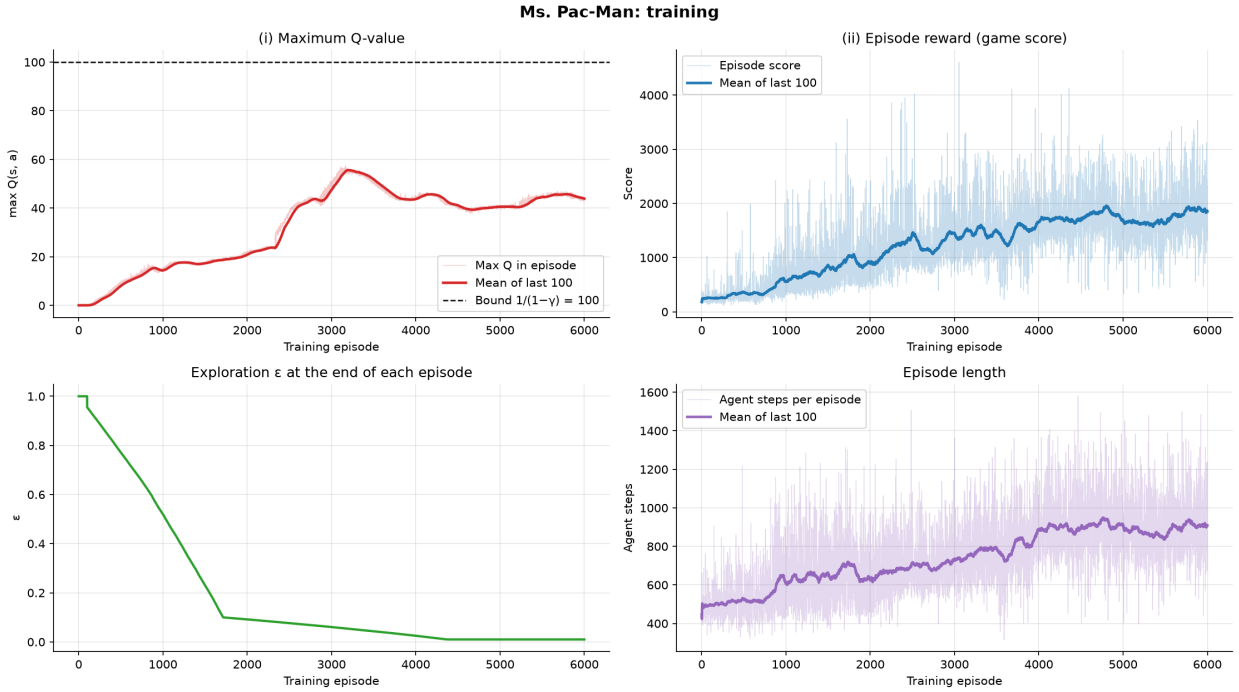


Figure 10: Training: (i) maximum Q-value per episode, (ii) episode reward (game score) with the moving average of the last 100 episodes, the exploration rate and the episode length.

- **The agent learns steadily:** the moving average rises from ≈ 250 (random play) to ≈ 1,850, and the mean score of the last 500 episodes is **1,848, compared with 296 for the first 500**. The best moving average is 1,958.
- **The games get longer:** from ≈ 500 to ≈ 900 agent steps, so the agent survives longer, not only eats faster.
- **Learning follows the exploration schedule:** the curve is almost flat while $\epsilon > 0.65$ (first ∼750 episodes), rises fastest while ϵ decays to 0.1 (episode ≈ 1,750), and flattens at about 1,700–1,900 after ϵ reaches 0.01 (episode ≈ 4,350).
- **(i) Max Q stays well below the bound of 100.** It rises with the score, overshoots to 57 around episode 3,200 and settles at 40–45. There is no sign of divergence.

Model Selection and Required Plot (iii)

As in part (a), the reported network is chosen on **validation games** (seeds from 10,000 on), separate from the 500 test games (seeds 0–499). Every snapshot (every 500 episodes) plays 30 greedy validation games, and the best one is evaluated on the test games.

(iii) Evaluation of the selected network (episode 4,000): mean = 1,790.6, std = 286.8 (Figure 12).

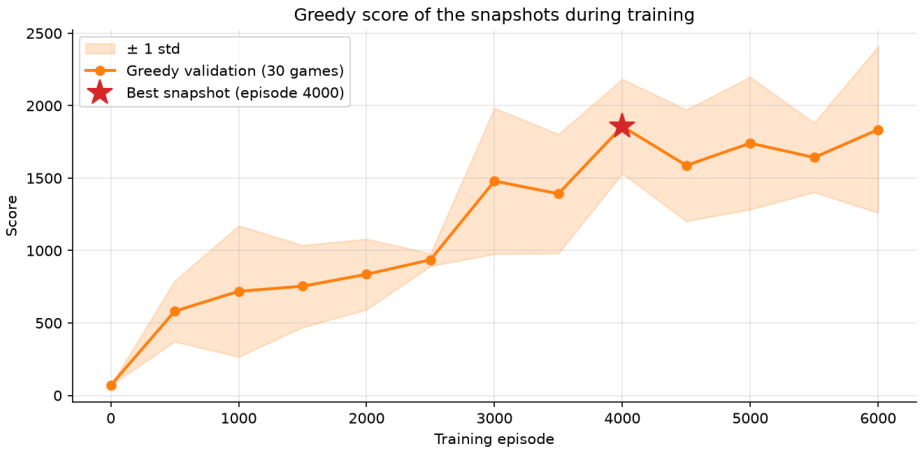


Figure 11: Greedy validation score (30 games) of the snapshots during training. The star marks the selected snapshot.

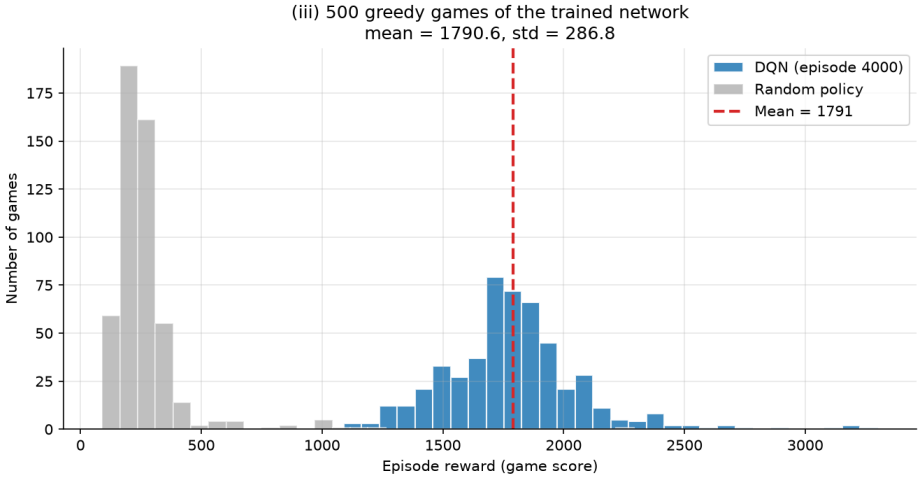


Figure 12: (iii) 500 greedy test games of the selected network (episode 4,000), with the random policy as reference.

Network (500 greedy test games)	Mean ± std	Median	Max	Mean length
Selected snapshot (episode 4,000)	**1,790.6 ± 286.8**	1,780	3,300	924
Final weights (episode 6,000)	1,842.7 ± 624.7	1,715	4,740	902
Random policy	266.2 ± 164.6	240	2,290	499

- **The trained agent scores 6.7× the random policy**, and the two histograms barely overlap.
- **Selected vs. final network:** on validation, episode 4,000 is marginally ahead (1,857 vs. 1,835), on the test games the final network has a slightly higher mean (+52, about 1.7 standard errors) but more than twice the spread. 30 validation games cannot separate them. I report the selected network, as fixed before looking at the test games.
- **After episode 4,000, the validation score plateaus** at 1,600–1,850 (Figure 11): more episodes at $\epsilon = 0.01$ do not help this configuration any more.
- **For orientation:** the original DQN paper (Mnih et al., 2015) reports 2,311 for Ms. Pac-Man after 50M agent steps, about 11× the training used here, with a slightly different evaluation protocol (no sticky actions).

Non-Standard Plot: Where Does Ms. Pac-Man Go?

Ms. Pac-Man's position is read from her colour in the raw frame. The heatmaps count how many steps she spends at each position over 10 greedy games, drawn over the maze.

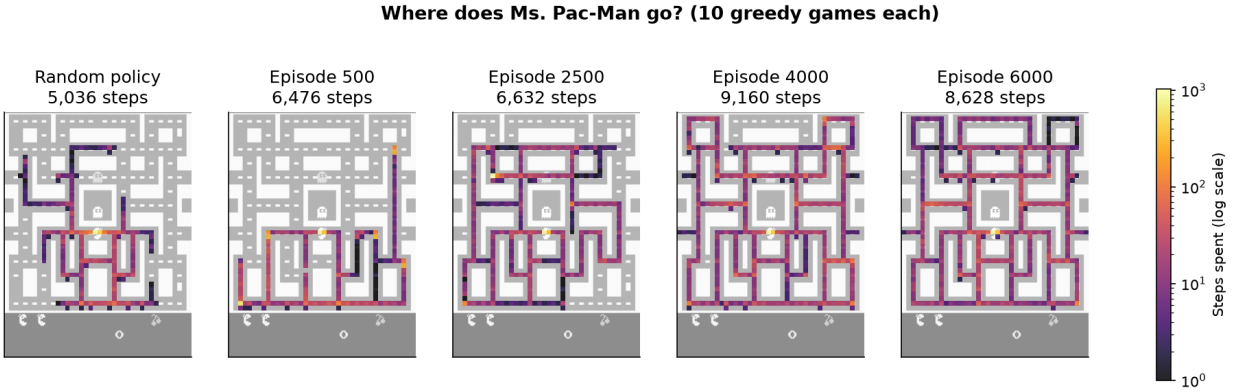


Figure 13: Ms. Pac-Man's position over 10 greedy games (log scale) for the random policy and four snapshots.

- **Random policy:** she stays around the start position in the centre and dies there.
- **Episode 500:** long routes, but only through the lower half of the maze.
- **Episode 2,500:** most of the maze, but the top corridors are still almost never reached.
- **Episodes 4,000 and 6,000:** the **whole maze**, including the top corridors and the tunnels at the left and right edge. The agent has learned to clear the board, not only to collect the dots near the start.

The second plot follows the Q-value through one greedy game of the selected network:

- **When play starts, Q jumps to its maximum (≈ 45)** and then **falls steadily as the dots are eaten**: fewer dots left means less reward to come. This is exactly what a Q-value should do.

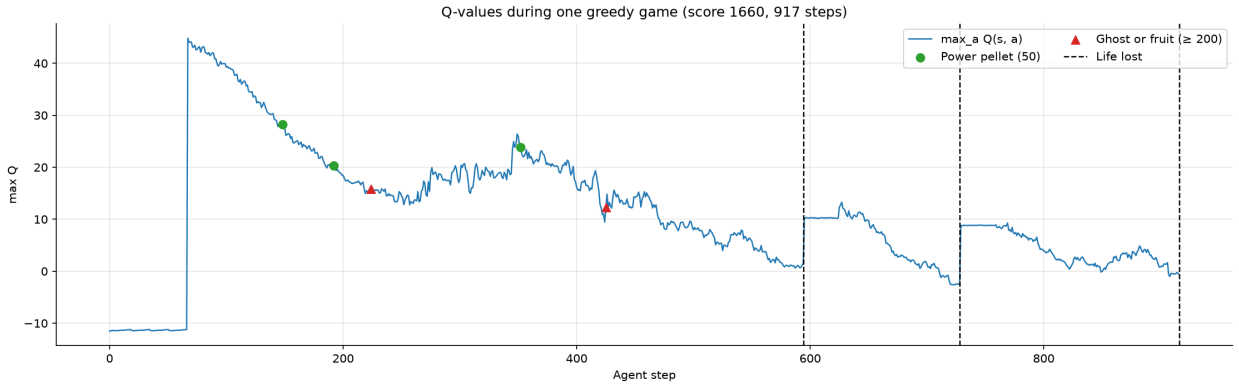


Figure 14: $\max_a Q(s, a)$ during one greedy test game (score 1,660), with power pellets, ghosts or fruit eaten, and lost lives.

- **A power pellet lifts Q** (step ≈ 350): eatable ghosts are worth extra rewards.
- **Before every lost life, Q drops to ≈ 0–2**: the network sees death coming. After the respawn, Q is flat at ≈ 10 during the pause, lower than at the start because fewer dots are left.
- **Unexpected: during the first 67 steps (the start delay, in which the game ignores all actions), Q is** ≈ -11, although all rewards are ≥ 0. This is examined in Test 6.

Edge Cases

Edge case	Handling and check
Sticky actions	Kept, as v5 prescribes. Even the greedy policy is therefore stochastic. Without them, every greedy game is identical (Test 9).
Flickering sprites	The ghosts are not drawn in every frame; with 4 stacked frames each ghost is visible in at least one (Figure 9).
Life loss vs. game over	A lost life is terminal for the TD target, but the game is not reset; only the third life ends the episode (Test 3 ✓).
Time limit	A truncated game is not terminal. Its last step has no stored next state, so it is never sampled instead of being treated as terminal (Test 2 ✓).
Frame stack at a game start	Filled with the first frame. In the buffer, stacks never reach back into the previous game but repeat the first frame, exactly like the online stack (Test 2 ✓).
Buffer wrap-around	A sample needs all 4 frames of its stack still stored and its next state already stored (Test 2 ✓).
Reward scale	Clipped to $\{0, 1\}$ for learning, raw score reported. Q is bounded by 100 (Test 6a ✓).
int8 arithmetic of the preprocessing	`img.sum` gives unsigned integers, so `img // 3 - 128` wraps around; the cast to int8 still gives the right value (Test 1 ✓).
Overestimation	Double DQN, slow target network, targets without gradients (Test 6).
Crash during the overnight run	Atomic checkpoints with the replay buffer and all generator states; per-episode seeds; deterministic GPU algorithms (Test 10 ✓).
Model selection on the test set	Snapshots are compared on validation seeds (10,000+), the test uses seeds 0–499.
Memory	1M frames as int8: 7 GB of RAM; the full buffer is loaded from a checkpoint without a second copy.

Tests for Validating Correctness

All tests use the selected network unless stated otherwise. Tests 1–3 check that the agent learns from the right data (the inputs, transitions and terminal signals the algorithm depends on), Tests 4–10 check the learning behaviour itself. Each criterion was fixed before the run. **23 of 24 checks pass.**

Test	What is checked, and why it validates the algorithm	Result
1. Preprocessing	Frame is 88×80 int8, equals a signed reference computation, Ms. Pac-Man's pixels become −128. *Why:* the int8 cast wraps around; a wrong sign would hide Ms. Pac-Man from the network.	3/3 ✓
2. Replay buffer	Small buffer (64 slots, 421 frames, wrapped around): sampled stacks of s and s' equal the online stacks, no stack mixes two games, truncated last steps are never sampled. *Why:* index-based stacking is the most error-prone part; a mismatch would train on states the agent never sees.	5/5 ✓
3. Life loss	A random game: 3 TD-terminal steps, only the last one ends the game, no reset in between. *Why:* the terminal signal decides where the TD target stops bootstrapping.	3/3 ✓
4. Learning	Last 500 training episodes > 2× the first 500; test score > 2× random. *Why:* the grading criterion; improvement must come from learning, not from chance.	296 → 1,848; 1,791 vs. 266 ✓
5. Progress	Validation score of the 13 snapshots rises with training (Spearman ρ); best snapshot > 3× the untrained one. *Why:* greedy play on fixed games, independent of ϵ and of the training noise.	$\rho = 0.93$; 70 → 1,857 ✓
6. Q vs. actual return	(a) max Q < $1/(1-\gamma) = 100$; (b) correlation of Q with the actual discounted return G along greedy games > 0.5; (c) Q in the 10 steps before a lost life < 80 % of its mean. *Why:* Q must estimate the return, not only rank actions.	(a) 57.3 ✓ (b) 0.46 **[FAIL]** (c) 4.5 vs. 12.5 ✓
7. $\gamma = 0$ run	Changed parameter: with $\gamma = 0$, Q must equal the probability of a reward, so all Q in $[-0.1, 1.1]$ and mean Q = reward rate. *Why:* tests the target without bootstrapping, where the right answer is known.	$[-0.03, 0.34]$; 0.053 vs. 0.052 ✓
8. First 10 steps	Q of all actions, action, reward and TD target step by step; mean |TD error| < 1 over a full game. *Why:* a trained network must be close to a fixed point of its own Bellman update.	0.48 ✓
9. Harder evaluation	Reduced and added stochasticity: 100 games without sticky actions (deterministic) and with $\epsilon = 0.05$, both > 2× random. *Why:* the policy must not depend on one lucky sequence of events.	1,710 ± 0; 1,782 ± 420 ✓
10. Exact resume	6 episodes in one go vs. 3 episodes, "crash", resume, 3 more: identical log and weights. *Why:* the reported run was resumable; a resume must not change the result.	2/2 ✓

- **Test 6 (Q vs. return):** unlike part (a), these roll-outs are greedy, so Q and G refer to the same policy. During play, Q follows G closely, but the start of play is clearly **overestimated** (44.8 vs. ≈ 27 in all 10 games). Over all steps outside the start delay, the mean Q is 14.4 vs. a mean G of 9.7.

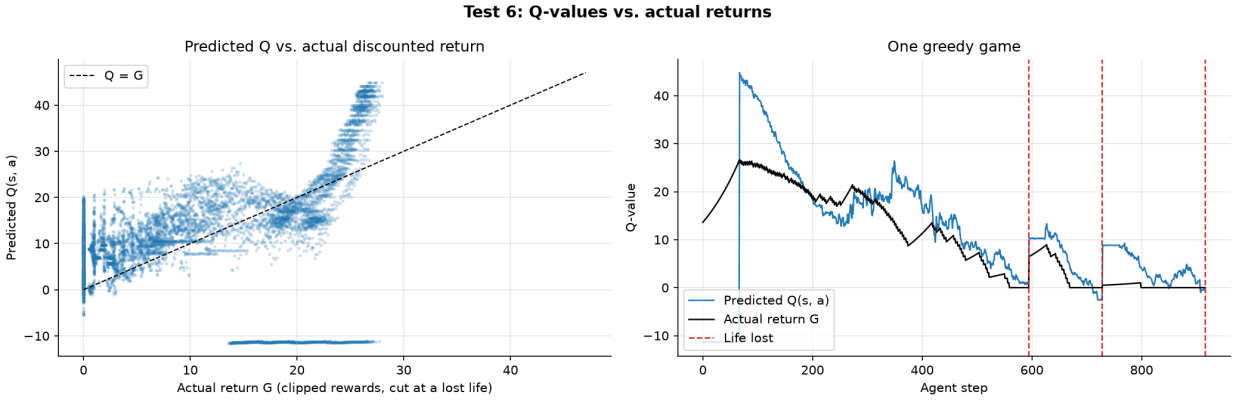


Figure 15: Test 6: predicted $Q(s_t, a_t)$ vs. the actual discounted return G_t of the clipped rewards (cut at every lost life) over 10 greedy test games (left), and along one game (right).

- **Test 7** checks the target computation in isolation from bootstrapping: Q matches the reward rate to 0.001.
- **Test 8:** the literal first 10 steps fall into the start delay: all 9 Q-values lie within 0.02 of each other (≈ -11.5), which is correct in so far as the action does not matter there. The table therefore also shows the first 10 steps after the delay, when Ms. Pac-Man starts to eat (test game 0; $y = r + \gamma \max_a Q(s', a)$ with the same network):

Step t	67	68	69	70	71	72	73	74	75	76
Action	UP	UP	UP	DOWN	DOWN	DOWN	UPRIGHT	DOWN	NOOP	NOOP
$\max_a Q(s_t, a)$	44.82	44.04	44.08	44.12	43.12	43.10	43.49	42.59	42.69	43.05
$Q(s_t, \text{LEFT})$	44.58	43.72	43.30	43.13	42.14	42.82	43.39	42.54	42.59	43.05
Raw reward	10	0	0	10	0	0	10	0	0	0
Clipped reward r	1	0	0	1	0	0	1	0	0	0
TD target y	44.60	43.64	43.68	43.69	42.67	43.06	43.17	42.26	42.62	42.77
TD error $y - Q(s_t, a_t)$	−0.22	−0.40	−0.40	−0.44	−0.45	−0.04	−0.33	−0.33	−0.07	−0.29

- **The Bellman update is visible step by step.** Whenever a dot is eaten ($r = 1$ at steps 67, 70, 73), Q drops by about 1 in the next step (44.8 → 44.0, 44.1 → 43.1, 43.5 → 42.6): one reward fewer is left to collect. Between two dots, Q rises again slightly as the next dot comes closer and is discounted less.
- **The TD errors are small but all negative** (−0.04 to −0.45): the network predicts slightly more than its own next-step target, the same overestimation at the start of play that Test 6 found. Over the whole game, the mean |TD error| is 0.48 ✓.
- **The policy decides on tiny margins.** At step 67, LEFT is only 0.24 below the best action; the median action gap over the game is 0.028. As in part (a), most decisions are close calls, which is why the dueling head (learning V separately) suits the game.
- **Test 9:** with 5 % random actions, the mean barely changes (1,782). **Without sticky actions, all 100 games are identical (1,710 ± 0):** the emulator is otherwise deterministic, so the whole spread of the evaluation comes from the sticky actions.

Why Test 6b fails: the start delay. At the start of every game, Ms. Pac-Man cannot move for 67 agent steps. In these steps, the network predicts $Q \approx -11$, although with rewards ≥ 0 every true Q-value is positive ($\approx 0.99^{67-t} \cdot 27$). These 7.6 % of the steps form the horizontal

band at $Q \approx -11$ in Figure 15. **Without them, the correlation is 0.80**, well above the threshold.

The delay states are not frozen frames: the ghosts move in a cycle of 16 steps, so the same stack appears four times at different distances to the start of play. The negative value appears in every snapshot and moves with training (-0.6 at episode 500, -11.5 at episode 4,000, -2.0 at episode 6,000), and the TD errors there are consistently positive ($+0.1$), so the update pulls it in the right direction but does not win against the generalisation from the rest of the network. I could not explain the cause with the remaining time. **It does not affect the score**, since the game ignores every action during the delay, but it is a real error of the value function.

Summary of Part (b)

- **Model:** Double DQN with a dueling head, 4 stacked frames, 1M replay buffer, slow Polyak target network, Huber loss with clipping and clipped rewards; 6,000 episodes (4.46M agent steps, 5.8 hours).
- **Result:** the selected network (episode 4,000) scores **1,790.6 ± 286.8** over 500 greedy test games, 6.7× the random policy (266.2). It clears the whole maze instead of staying near the start, and its Q-value reacts sensibly to dots, power pellets and approaching death.
- **Validation:** 23 of 24 checks pass, including a bit-identical resume after a simulated crash. The failed check reveals a negative Q-value during the start delay, which does not affect play. Apart from it, Q follows the actual returns ($r = 0.80$) but overestimates them, especially at the start of play.

4 Given an Additional 3 Months, What Other Validation Tests and Experiments Would I Perform?

CartPole (a)

- **Fix the inaccurate Q-values near failures** (Tests 1a and 6): prioritised replay, keeping failure transitions in the buffer longer, or training from wider and perturbed start states. Then check whether Q one step before a failure drops to ≈ 1.
- **A cleaner calibration test:** add random actions only in the first steps and then act greedily, so that the measured return is the one Q actually estimates.
- **Stable training:** the policy still collapses briefly during training. A learning-rate scheduler, a slower ϵ decay, or a larger batch should show whether the moving average can stay above 475.
- **More seeds (e.g. 10)** for every variant of the ladder, to separate real effects from noise (e.g. Double vs. vanilla DQN, or the drop at v3).
- **Remove the time limit** (5,000 or more steps) and add disturbances (random pushes, observation noise, other pole lengths), since the standard evaluation is saturated.

Ms. Pac-Man (b)

- **Find the cause of the negative Q-values in the start delay** (Test 6b): train a run in which the delay steps are skipped or excluded from the buffer and check whether the rest of the value function changes; inspect V and A of the dueling head separately.
- **A hyperparameter ladder, as in part (a),** with several seeds per variant: buffer size, τ, learning rate and the ϵ schedule. Ablations of Double DQN and the dueling head would show which component matters here; with one seed, I can only show that the combination works.
- **Train longer and stronger:** the score plateaus after episode 4,000. Prioritised replay, n-step returns and distributional or Rainbow-style DQN are known to raise Ms. Pac-Man scores a lot; 50M agent steps as in the DQN paper would show whether the plateau is the method or the budget.
- **Reduce the overestimation at the start of play** (Q 45 vs. an actual return of 27) and check whether a better-calibrated Q also plays better.
- **Harder and broader evaluation:** other game modes and difficulties of `MsPacman-v5`, the later mazes (does the agent ever clear the first board?), and the generalisation to unseen start conditions such as random no-op starts.
- **More analysis of the behaviour:** how often the agent chases eatable ghosts after a power pellet, and where in the maze it loses its lives.